8 January 2021

COMPUTER PROGRAMMING level 4

PROGRAM: is a step by step of instruction for telling the computer how to perform an operation and accomplish a desired task.

- A program is also known as the **code**.

PROGRAMMING: The act/ craft of designing and developing computer programs

- the process is also known as **programming**.

PROGRAMMER; Is a person or an individual who has got the expertise of designing and developing computer programs.

- also known as a **CODER.**

PROGRAMMING LANGUAGES- Is an artificial language containing words characters/symbols syntax and semantics for developing computer programs.

EXAMPLES OF PROGRAMMING LANGUAGES

Java, python and c++ and many more.

11 January 2021

SYNTAX: Are rules governing how words/characters in a programming language should be combined in older to form an acceptable programming statement.

SEMANTICS: Are meanings of words/ characters that are used in a programming language.

TYPES OF PROGRAMMING LANGUAGES

Programming languages are classified based on:

- Proximity to natural languages
- How they get translated
- Paradigm (approach/ technique)

NB: *Natural languages are languages that are spoken by man.*

Proximity *simply means how close the languages are from the real natural languages that are used by man. They are further classified int two namely* ***high-level languages*** *and* ***low-level languages.***

1. **HIGH LEVEL LANGUAGES**

- This are languages that are close to natural languages (*uses symbols and words that are understandable and used in natural languages*)

  EXAMPLES: c++, java, python, and visual basic.

2. LOW LEVEL LANGUAGES

- These are languages that are close to the **native language** of the computer.
- The native language of the computer is the **binary language** which uses two characters [zeros and ones (1&0)] to form a string of information that the computer understands.

  EXAMPLE: (*101010110100*10*) >> the string of information.

14 January 2021

TYPES OF CODES IN PROGRAMMING

1. SOURCE CODE

- This is the type of code written by programmer in any high-level language before translation into machine cade/ object code.

2. MACHINE CODE

- This is the code which is in the binary language of string of zero and ones (Os, 1s) > it's a product of translation.

  Bytecode -

  What is Translation?

- Is the process of converting source code into machine code.

CODE TRANSLATOR

- A translation is a piece of software that is used to convert source code into object code. (machine code)

Byte= 101010 > collection of 8 bits

1byte=8bits

Kilobyte=1000(1024)

TYPES OF CODE TRANSLATION

Code translation get classified based on the types of translation which is used for a particular language

TYPES OF TRANSLATOR

1. COMPILERS

- Is a translator which converts source code by reading all the lines of code and transforms them all into objects code at once.

Compilation => Is the process by which source code gets converted into object code by using a compiler.

- Programs that come from compilers are called **compiled programming languages**

2. INTERPRETERS

- An interpreter is a code translator which converts source code by reading one line at time and generate a result (0s, 1s) until it reach at the end the last code statement in a source code file.

27TH JANUARY 2021

TWO MAIN TYPES OF PROGRAMMING LANGUAGES

- Depends on how the source code written in that language gets translated.
- All languages that source code to be translated with the use of the compiler are called ***compiled programming languages.***
- If language requires the source code to be translated using an interpreter then it is called ***interpreted programming languages.***
- Languages are classified with respect to the type of the translator.
- Depending on the nature of the program been developed, the platform where to run and the execution speed you will enable whether to use a complied language programming or interpreted programming language.

EXAMPLES OF COMPILE PROGRAMMING LANGUAGES

- C++
- C
- C sharp

- Visual basic
- Cobal
- Java (also interpreted programming language it is hybrid in a way)

EXAMPLES OF INTERPRETED PROGRAMMING LANGUAGES

- PHP
- Python
- Java script
- Ruby on rails
- VB script
- Java saver pages

CHARACTERISTIC OF COMPILED PROGRAMMING LANGUAGES

1) The object code (*end product code*) is in zeros and ones (*machine code*)

- After loading the source code through all 100 lines from 1 to 100 depending how many your source code has and it has no mistakes, no syntax errors, no grammatical errors then the compiler will Covent the set of instruction into machine code. Machine code produced is permanent.

2) Programs generated are speed critical.

- There is fast execution relative to programs that are interpreted in nature.
- They execute (run) with speed since they instruction are in zeros and ones which is the native languages of the computer hence the computer does not have to translate again and will simply go to the language and run it hence the execution is fast.

E.g. machine used in hospital theaters.

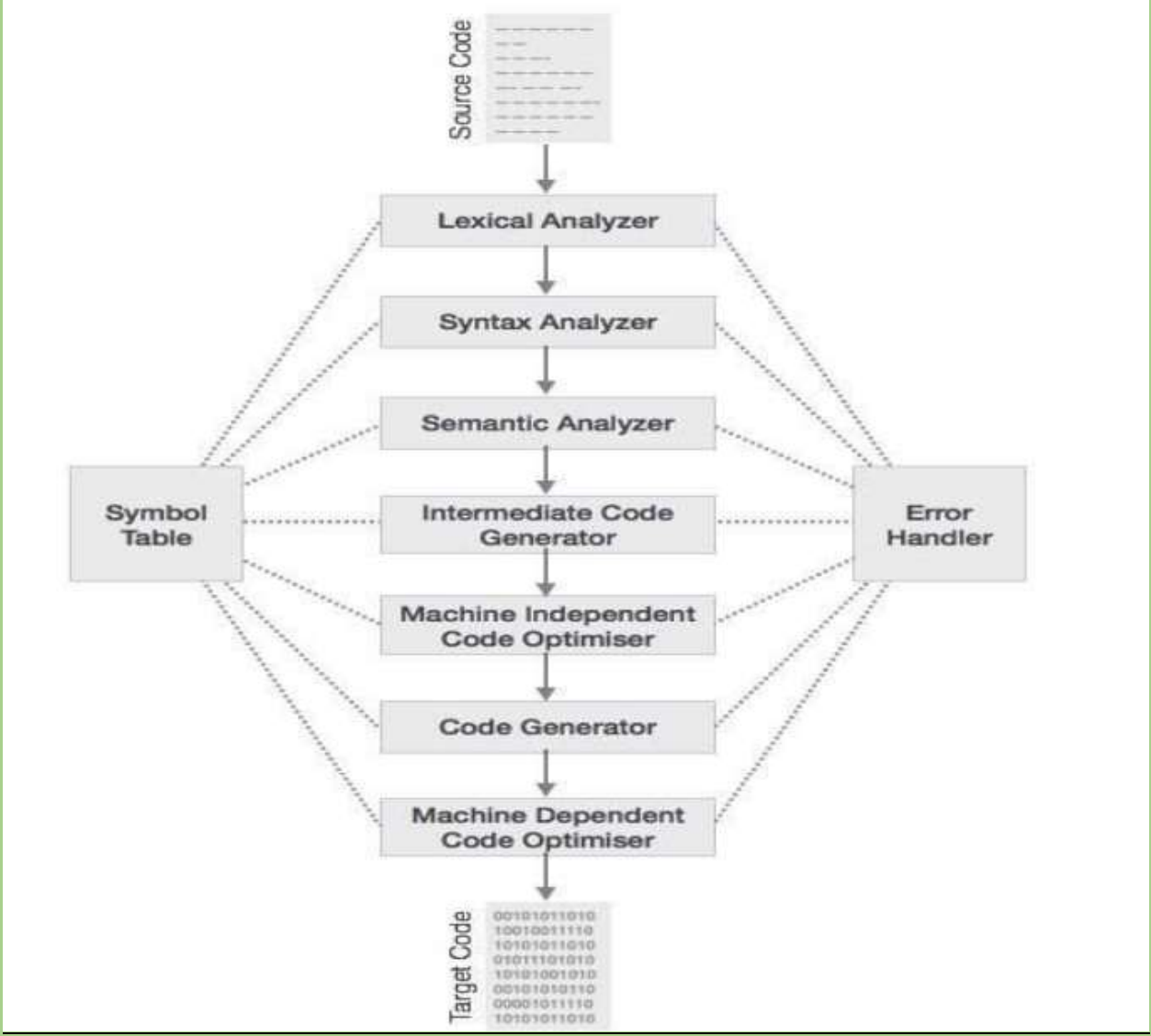
3) Once the program is compiled it **can be used many times**. No need for recompiling of the program.

DISADVANTAGES OF COMPILED PROGRAMMING LANGUAGE

1. Are machine dependent:
   - Once the program is compiled on a particular machine processor (machine CPU), it takes the form of the CPU and if you're to execute it on another machine with a different CPU structure or hardware structure, the program will not respond well or may respond at all hence machine dependent.
     (i) Complex instruction set computing processors (**CISC**)
     (ii) Reduced instruction set computing processors (**RISC**)

NB: If you compile your program on the RISC processors, it may not respond well or it may not respond at all on a CISC processors and Vice-versa.

COMPILATION PROCESS IN JAVA.



Lexical analyzer => the source code gets scanned and then gets converted into lexemes.

Syntax analyzer => the lexemes produced are taken as inputs and generates a syntax tree where the lexeme arrangement is checked against source code grammar.

Semantic analyzer => checks whether the syntax tree constructed follows the rules of the language and also keeps track of identifiers their types and expressions.

- At the end, semantic analyzer produces an annotated syntax tree as an output.

Intermediate code generator => it generates an intermediate code of the source code for the target machine.

- It represents the program for some abstract machine and it is in between **the high language** and **the machine language**
- The intermediate code is generated in such a way that it is very easier to translate it to targeted machine code.

Machine independent code optimizer => the intermediate code is now checked [to remove unnecessary lines on code and arranges the sequence of statement in order to speed up the execution of the program without wasting memory.]

Code generator => the optimized intermediate code is now mapped to the target machine language.

- The code generator translates the intermediate code to a sequence of machine code. [sequence of instructions of the machine code]

CHARACTERISTICS OF INTERPRETED PROGRAMMING LANGUAGES WITH RESPECT TO THE END PRODUCT OF INTERPRETATION

a) With the interpreter **translation of source code is done line by line** and results are generated for each line and with this interpreted programming languages execute (run) slowly since each line has to be read and decoded and results are given and therefore a lot of time is involved to read decode and generate results.

b) They are **easy to write**: - They do not demand strict adherence to syntax, when writing the source code, you can violate a few things and the program will still run.

c) It's **easy to identify code errors**: - Since the interpretation happens line by line, so when an error is discovered in a line, the interpreter does not continue translating, therefore it stops and the error is easily located.

   - Easy to debug when using interpreted program languages.

d) The end product is not an object (a program) code which is permanently in the machine code in zeros and ones because the interpretation is line by line thing and at the end of the day you do not have a package of Os and 1s.

f) **Reinterpretation is done =>** When it comes to executing the program, each time you want to use the program you have to re-interpret it, retranslate it because it's not permanently in the machine code.

g) **Machine independent =>** Programs that comes from interpreted programming languages are machine independent.

   - They can run on any operating system platform whether android, Linux, windows or mac OS.

NB: in conclusion if you are looking for a program and your interest is that the program should be able to run in any hardware platform E.g.

internet-based programs, therefore your program must be developed using an interpreted programming language.
If your interest is to develop a program which must execute relatively faster, than compiled programming languages must be used.

5th February 2021

THE ANATOMY OF A JAVA PROGRAM

- ✓ the anatomy of a java program has the following syntax

```
Public class ClassName {
        Public static void main(string []args) {
        // Code to run
        }
}
```

JAVA PROGRAMMING LANGUAGE

Is an object oriented programming language

- ✓ For one to come up with a program in Java one must come up or create an object first which will send in errands to do the tasks and provide the services the program is supposed to be providing.

- ✓ Functionality of the program are provided by object.

Programs are developed as a set or package of objects that are told what to do.

- ✓ Objects: based what you know in real life e.g. stone and phones.

One must focus on developing objects that can be told what to do which are the services that the end user is looking for.

FUNDAMENTALS OF A JAVA FILE

Public class ClassName

- **Public**
  - Is also known as the access modifier, giving access to anyone.
- **Class**
  - Object are put classes based on their functionalities
  - Object have attributes (*specific properties*).that are used to classify objects.
  - Is a way of telling the computer that you are developing a class which will contain objects of the same attributes which will coming up with a class?
  - A class is key word which specifies that you're opening up a class.
- **Keyword**

- Is a special word in a programming language which has a unique meaning so that it can only be used according to the meaning it possesses in the language and not otherwise.

Saved for notes

1) **The JOoptionPane Method**
    - Is a graphical used for capturing input in Java using inbuilt swing, methods

      NB: JOptionPans class can also be used to handle outputs.

HOW TO USE THE JOptionPane CLASS

- use the following import statement to import the class into your program
    - import Javax.swing.JOptionPane;

NB: The statement name above must be written just b4 the class declaration statement.

2) **Use the input dialog box to display the input window using the following statement.**

THE CPU ARCHITECTURE & THE FETCH, DECODE EXECUTE CYCLES

KEY FUNCTIONS (*OPERATIONS*) OF THE COMPUTER

(i) Capture input
(ii) Process the captured data /store
(iii) Manage the captured data
(iv) Manage storage

HARDWARE PARTS OF A COMPUTER

(i) **The processing hardware** [the CPU]
(ii) **Peripheral** = hardware components connected the CPU through various networks in the computer.

THE CPU

Central processing unit (CPU) it is the heart of the computer where processing of data is performed.

PARTS OF THE CPU

A. The Control Unit (CU)
B. The ALU (arithmetic & logical unit)
C. Register

1. THE CONTROL UNIT

- The control is the main hardware component of the **CPU** responsible for all manner of computation/ calculation which the CPU undertakes.

FUNCTIONS OF THE CONTROL UNIT

- The CU is responsible for the flow of Data and Instructions from memory into the CPU where processing is done.
- The CU is responsible for the movement of data (after processing) from the CPU to memory and further storage units. (HDD)
- The CU coordinates the buffering of data in register associated with the ALU and the CU itself.
- The CU coordinates the buffering of the instructions.
- The CU is responsible for the decoding of the instructions in the processor.
- The control unit is responsible for the execution of instructions.

THE CPU CYCLE

THE FETCH-DECODE-EXECUTE CYCLE

- ✓ Continually (repeatedly the processor specifically inside the CU data instructions undergoes 3 stages as long as the computer is in use)
- ✓ The 3 stages of processing gets refried to as the cycle or the fetch decodes execute cycle.

1) FETCH

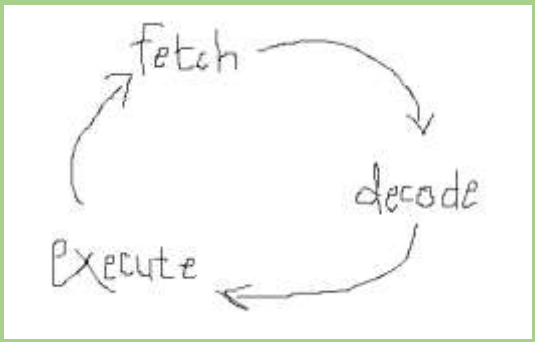
- ✓ Instructions and data are fetched from the memory and buffing registers to the processor.

2) DECODE

- ✓ The instruction are interpreted (translated) according to the architecture of the CPU.

3) EXECUTE

- ✓ The instructions are then executed e.g. applying them on data to generate demanded results.



The processor all the time goes through the stages above in order to process information.

4) THE ALU (Arithmetic logic unit)

- ✓ Arithmetic: is responsible for all mathematical computation in a processor
- ✓ Logic: Is responsible for all decision-making processes in a processor

5) THE REGISTER

- ✓ Is the high speed memory close to the CPU responsible for transmission of data from and out of the CPU

TYPES OF REGISTER

(i) Memory address register (**MAR**)

(ii) Memory data register (**MDR**)

- Responsible for buffing instructions for the process only

(iii) Current restriction register (CRR)

PROGRAM FLOW CONTROL IN JAVA

- Is an idea which describe how programs are designed to execute or to flow. Certain techniques known as programming **CONSTRUCTS ARE** used to structure up how a program can flow or execute.

TYPES OF PROGRAMS BASED ON CONSTRUCTS

- ✓ Sequential
- ✓ Selective or conditional (*if statements and others*)
- ✓ Interactive or looping (*while, for loops*)

SEQUENTIAL PROGRAMS

- A sequential program is the one in which every single line of code in the program gets executed during RUN – TIME hence no line pf code is left un-executed.

SELECTIVE PROGRAMS

- Also known as conditional programs: this are programs in which a block of code are executed **OR NOT** based on the condition being **TRUE OR FALSE.**
- Selective programs use the section process to select the block of code to execute or not.
- In the selection construct the following statements are used:
    a. If ----statement
    b. If---else—statement
    c. If---else if--else--statement

a. IF STATEMENT

- If selection uses the **if** keyword

SYNTAX:

If (condition) {

// Code to execute if the condition is true

}

NB:

A. The code in the block will execute only when the set condition is true.
B. The condition is given as the set of :

- Test variable (*variables whose value will be tested*)
- Relational operation
- Value to be tested

RELATIONAL OPERATORS (*inbuilt piece whose value will be tested*)

OPERATOR SYMBOL	NAME
>	Greater then
>=	Greater than or equal to
<	Less than
<=	Less than or equal to
==	Equal to
!=	Not equal to

EXAMPLE:

```
Double grade;

grade=52.8;

If (grade>50) {

System.out.println("COGRATS YOU HAVE PASSED");

}
```

9TH March 2021

b. THE IF … ELSE STATEMENT

- Is a selective program methodology for implementing selection in which if a SET condition is FALSE the program shunts/ stops and executes another block of code found in in ELSE.

SYNTAX:

```
if (condition) {
```

```
//Code execute if condition TRUE
}
else {
//Code to be execute if condition is FALSE
}
```

EXAMPLE:

```
Double Grade = 57.3;
If (Grade < 30) {
System.out.println("you have FAILED");
}
else {
System.out.println("YOU HAVE PASSED")
}
```

c. THE IF... ELSEIF... ELSE

When creating a range, we use the ampersand or two ampersand sign &&

EXAMPLE:

FAIL

BARE PASS

CREDIT

DISTINCTTION

```
Int grade;
```

If (grade>=0 && grade<=39) {

}

else if (grade>=40 && grade<=64) {

Is used when multiple conditions are to be tested

SYNTAX:

If (condition1) {

// Code to execute if condition 1 is true

}

Else if (condition2) {

// Code to execute if condition 2 is true but condition 1 is false

}

.

. This dots are called ellipsis this means the codes are continuing

.

Else if (condition X) {

//Code to execute when all condition above condition X are false out condition X is true

}

Else {

Code to be executed when all conditions have felled

}

ITERACTIVE PROGRAMS

Also known as looping programs.

- An iterative program is in which a block of code will be executed multiple times as long as the set condition is true.

  NB: when a set condition is false or becomes FALSE the program:
  - ✓ Will not execute
  - ✓ Stops executing

MAIN TYPES OF LOOPS

- ✓ **Infinite loops:** this are loops whose condition is always TRUE hence cannot exit (*will not stop executing*)
- ✓ **Finite loops:** this are loops that have an exit point (*stops executing when the condition has become FALSE*)

SUB TYPES OF LOOPS

- There three main sub type of loops:
  - ✓ **The while loop**
  - ✓ **The for loop**
  - ✓ **The do-while loop**

THE WHILE LOOP

- This type the WHILE keyword is used to implement a loop

  SYNTAX:

  In general

  While (condition) {

  //Code to execute several times as long as the condition is true

  }

EXAMPLE:

Program to display the first 11 numbers (i.e.0-10)

SOLUTION:

- Determine the entry point into the loop --- starting point
- Determine the exit point ---- (*where to stop executing*)

CODE:

```
Int num = 0;
While (num<=10) {
System.out.println(num);
Num++;
}
```

NB:

- ✓ ++ In programming is an increment operator steps up (increases) the value in a variable by 1 each time a piece of code is executed. How is it used? For example when one has a value x the increment operator comes in front X++.
- ✓ -- is a programming is called a Decrement operator. It operates by decreasing a value in a variable by 1

CODE:

```
Int two =1;
While (two<=12)
{
System.out.println(two+''*2=''+'' ''+two*2);
Two++;
}
```

THE FOR LOOP

- Is a loop which uses the FOR keyword for its implementation.

  SYNTAX:

  For (initalValue: Condition;PostExpression)

  {

  Code to execute multiple times as long as the conditions is true

  }

  NB:

  IntialValue: is an initialized variable that contains an entry point value into the loop and is also called the test variable

  Condition: is the condition to be tested whose **TRUE** or **FALSE** status will determine the flow of the program.

  Postexpression: implements what must happen after the code in the **FOR** block is executed.

  Usually the **Postexpression** takes the form of an increment or decrement operator.

  EXAMPLE:

  ```
  Int a;
  For (a = 0; a <= 10; a++) {
  System.out.println(a);
  }
  ```

THE DO WHILE LOOP

SYNTAX:

```
Do {
Code to execute
}
While (condition);
```

Do while syntax uses two keywords:

- ✓ **Do** keyword
- ✓ **While** keyword

The code in the while loop will not execute if the condition is false while in do while loop the condition will execute once because the code executes before the condition.

GUI APPLICATIONS IN JAVA

DESIGN:

STEPS:

1) Create a project—file
2) New project

3) Java application

4) Give your project a name – GUIAPP

5) Uncheck the create main class checkbox

6) Click finish

7) Go to the project panel or window

8) Identify your project by name (**project you are working on**)

9) Click on source packages

10) Right click on default package

11) In the window which appears choose new

12) Then go to Jframe form in the next window

13) Click finish

14) Populating a form with controls

- ✓ Go into the palette window
- ✓ Pick a control of interest
- ✓ Drag it into the form
- ✓ Release the button to the control position on the form

FUNCTIONS IN PROGRAMMING

- Function is a module of code for implementing a specific task

Or

- Is a block of code for implementing a specific task.

Or

- Is an independent block of code for actualizing a desired service in a program.
- Function is also known as method procedure and subprogram.
- Function is small program inside of big program.

IMPORTANCE OF *VOID* FUNCTIONS

Used for:

- ✓ Generating prompt messages → prompt the end user to provide input to the system e.g. password, user-name
- ✓ Generating warning messages.
- ✓ Generating system status messages.
- ✓ Generating general informative messages.

EXAMPLE OF A VOID FUNCTION:

a. A function to delete a file
b. A function to modify/ manipulate a file.
c. A function to add two number

PARTS OF FUNCTION

a. a function has two main parts

1. **THE HEAD**
   - Contains the declaration statement.
2. **THE BODY**
   - is the code which actualize the desired objective of the function.

MAIN TYPES OF FUNCTIONS

1) *IN-BUILT FUNCTIONS*

- Also known as PREDEFINED FUNCTIOS
- This are functions that are built in a language and are used according to their design goal.
  - ✓ Main ( )
  - ✓ Println ( )
  - ✓ Print ( )
  - ✓ Next ( )
  - ✓ NextInt ( )
  - ✓ Next Double ( )

NB: any Java method terminates with a round brackets

2) *USER- DEFINED FUNCTIONS*

- ❖ These are functions that have NO PREDEFINITION in a programming language.
- ❖ These are functions written by a coder

DECLARATION OF FUNCTIONS

General syntax given as follows:

AccessModifier Returndatatype MethodName (parameters) **{**

//Code written here

}

WHERE:

1. **ACCESS MODIFIER.**

- Is a keyword which specifies the accessibility of the function e.g. public, private.

2. **RETURN DATA TYPE**

- specifies the type of data that a function will have when return back execution power to the caller function e.g. int, double, float, string, byte, char.

3. **METHOD NAME**

- Any user defined function is supposed to be given a unique name which reflect what the function will implement.

4. **PARAMETER**

- List of local variables used as storage area of data required by the function during execution.
- Is a variable which is associated with function.

EXAMPLESS OF FUNCTION DECLARATION

Public static int add (int x, int y)

NB: a user defined function is written

- Inside the class
- Outside the main ( ) method

EXAMPLE:

A function to add two numbers

Code

Public class addition {

```
Public static void main (string [ ] args) {

AddtwoNumbers (20, 30);

}

Public static int addtwoNumber (int num, int name) {

Int add;

Add = num 1 + num 2;

System.out.println(add);

Return 0;

}
```

- A function is called when it name is mentioned in another function which is executing.

NB:

- As the function is being invoked (called) arguments are passed to parameters of function.
- Arguments are actual values which are passed to parameters of function.

THE RETURN STATEMNT

- Return is a keyword which specifies
  - Completion of execution of a method
  - The value which a method most pass on to the caller method when returning back execution power/control e.g.

```
Public static double heightcalc (double x, double y) {

Double height = (x+y)/2)
```

System.out.println (“THE HEIGHT OF THE FIGURE IS” + height)

Return height;

}

- Return statement is a statement in a function which specifies execution completion of the function or the value which is supposed to pass on to the caller function.

NB: the return statement is written as the very last statement in a method body.

WHEN DOES A FUNCTION RETURN??

A function return when:

- It reaches the return statement
- It has reached the closing curled bracket of the function body.

THE RETURN VALUE

- Is a value which the invoked function carries to the caller function when returning.

VOID FUNCTIONS

- Is a function which does not return anything to its caller function i.e. when return it does not carry any data to give to the function which invoked it.
- Void function are specified by the keyword void.
- The old keyword is placed where the return datatype is written in the declaration statement of a method
  e.g. public static void main (string [] args)

PROPERTIES OF VOID FUNCTIONS

- Declared by using the **void** keyword.
- Has no return statement.
- May or may not contain parameters.

IMPORTANCE OF VOID FUNCTIONS

Used for:

- Generating prompt messages
- Generating warning messages
- Generating system status messages
- Generating general informative messages

Example of void function

```
Public static void ENTERNAME ( ) {

Public static void main (string [ ] args) {

ENTERNAME ( );

}
```

RECURSION

```
Public static void x (int y)

If (y = = 0) {

Return;

}

Else{
```

```
System.out.println (y)
X (y-1);
}
}
Public static void main (string [ ] args) {
X (10);
}
```

RECURSIVE FUNCTIONS

- Recursive is about function which call itself to execute.
- Recursive is a programming technique in which a function makes a call to itself.

```
Public static void ENTERNAME ( ) {
System.out.println("ENTER YOUR FULLNAME BELOW");
}
Public static void main (string []args) {
ENTERNAME ( );
}
```

ASSIGNMENT:

Using void functions create a banking program which prompts an end user to enter a 4 digit pin, if the pin is correct, it must DISPLAY a

service List and ask the end user to choose how he/she wishes to be served. Further the program must process the user choice and DISPLAY the system status message. TRANSACTION SUCCEFUL

DATA STRUCTURES

Also known as collection structures.

A data structure is an organized way of storing data in order to achieve:

- fast insertion
- fast retrieval
- fast modification
- fast deletion
- fast traversal

EXAMPLES OF COMMON DATA STRUCTURES

i. Arrays
ii. Linked lists
iii. Queque
iv. Stark
v. Tree
vi. Maps
vii. Dictionaries

ARRAYS (*array data structures*)

- An array is a contagious memory which stores up data of the same type.

Diagrammatic example:



Or



TYPES

DECLARATION OF ARRAYS

Int x [];

X=new int [6];

The above statement can be combined to form declaration of name and instantiation as follows:

Datatype arrayName []=new int [arraysize];

e.g.

int x [] = new int [6];

INTIALIZING ARRAY ELEMENTS

Array elements are initialized by assigning values into individual elements as follows:

Element = value;

e.g.

string Firstnames [] = new String [5];

Firstname [0] = "JOHN";

Firstname [1] = "JACOB";

Firstname [2] = "JAMES";

Firstname [3] = "JOEL";

Firstname [4] = "JONATHAN";

OUTPUT

Firstnames

JOHN
JACOB
JAMES
JOEL

JONATHAN

INITIALIZING of arrays can also be achieved as follows:

Datatype ArraysName[]={values1,value2,.,.,.,valueN}

EXAMPLE:

String Firstnames [] = {"JOHN","JACOB","JAMES","JOEL","JPNATHAN"};

This method involves declaration installation and initiation being done simultaneously array.

DISPLAYING VALUES FROM ELEMENTS

This is done by writing the name of the array element inside on output stream

e.g. System.out.println(Firtsname[0]); result = will display JOHN

1. System.out.println(firstnames.length) this code is used to check the number of arrays in the program.

The length is an input method for declaration of the size of the array.

TRAVERSING THROUGH AN ARRAY AND TO DISPLAY ALL THE VALUES

We use A FOR LOOP as given in the following example:

For (int I = 0 ; I < Firstnames.length ; I++) {

System.out.println(Firstnames[i]);

}

Example 2:

```
Int Age [] = {20,10,30,40,60,7};
```

NB: numeric datatype is not placed In quotes while character (char) datatype is placed in in double quotes.

ADDITION OF ALL VALUES (NUMERIC VALUES) IN ARRAY

Addition of all values in an array is done by using a FOR LOOP EXAMPLE:

```
Int Sum=o;
For (int i=O; i<Age.length; i++) {
Sum = Sum+Age[i];
}
{
System.out.println(Sum);
}
```

DETERMINING THE LAGEST VALUE IN ARRAY

Assuming: set up a sample array

```
Int x [] = {32, 40, 8, 60, 100, 20, 15,500};
Int Max;
Max =x[O];
For (int I = O ; I < x.length ; i++) {
If (x[i]>max) {
Max= a [i];
```

```
}
}
System.out.println ("THE LARGEST VALUE IS :"+max)
}
}
```

TWO DIMENSIONAL ARRAYS

- These are arrays that span in two dimensions(i.e. the x and y plane)
- With 2-D arrays:
  - Rows span downwards → in the y →plane
  - Columns span horizontally → in the x → plane

LABELLING OF A 2-D ARRAY

2-D ARRAY SIZE:

- The 2-D array size is given in terms of the number of rows and the number of columns

  E.g.

  3x3→ row x column → read as 3 by 3 array → 3 rows and 3 columns

  4x7 → 4 by 4 || →4 rows and 7 columns

NAMING 2-D ARRAY ELEMENTS

- Uses the array name and row position plus column position of the element

  e.g.

  syntax:

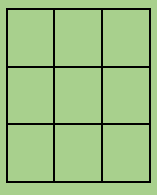
  2-D ARRAYS DECLARATION

  Datatype Name[][] = new datatype [row size] [column size];

  Int x [] [] = new **int**[3][3];

  Result:

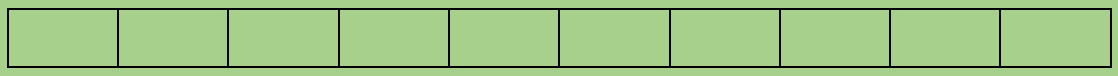
  Creates a 3x3 array called x→



Class work

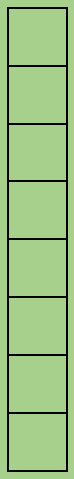
i. **Int** x [] [] = new **int** [1] [10];

Result:



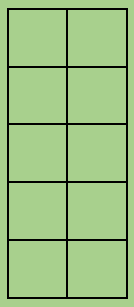
ii. Int x [] [] = new int [10] [1];

Result:



iii. Int x [] [] = new int [5] [2];

Result:



1. Consider the following 2 –arrays

X

5	10	35	100	-50	12	0

X1

PROJECTS THIS WEEK

ADVANTAGES OF ARRAYS

- Allows storage of data of the same type

DISADVANTAGES OF ARRAYS

- Cannot be used to store different type of data
- It is a static data structure →has a fixed size once declared.
- We need to know in advance the number of elements for an array

THE LINKED LIST DATA STRUCTURE

A linked list is a non -contiguous element data structure which store up data in storage areas called NODES.

- ✓ An element in a kinked list is called a NODE.

Diagrammatic representation

NATURE OF A NODE

A linked list NODE is made up of two parts.

i. The Data part
ii. The address part (*is also known as the pointer*)

As follows

Drawing……

- The **data** part is used for storage of data
- The **address** part contains the address of the next Node memory
- The first Node is called the HEAD
- The last NODE is called the TAIL

IMPLEMENTATION OF LINKED-LIST IN JAVA

>> linked lists in java is implemented as a string object.

>> A linked list class provides methods; -

- add first ()

- add last ()

>> for use with linked list the following class has to be imported into the program.

syntax;

```
LinkedList<String> ObjectName = new LinkedList<String>();
```

example.

```
import java.util.LinkedList;
public class MgLL {
    public static void main(String[]args){
```

```
LinkedList<String> Records = new LinkedList<String>();

Records.add("TINASHE");

Records.add("MWANGUULA");

Records.add("22");

Records.add("FEMALE");

Records.add("99000.22");

Records.add("CHRISTIAN");

Records.add("P");

}

}
```

STARK DATA STRUCTURE.

- It is linear data structure
- uses the LIFO (Last in First Out) principle as the last item to be taken into the DS is the First to be retrieved.
  => the first data item to be loaded/ taken into the stark is at the bottom
  => the last data item to be loaded/ taken into the stark is at the top.
- Data loading and retrieval is done at the same end point the open-ended point.
- Uses two main operations / method : -
  - Push (); => To insert data into the stark
  - Pop (); => To remove data from the stark.
  - The basic concept of stark is that it uses the push method to insert data and it uses the pop method to remove data from a stark



QUEUED DATA STRUCTURE

➢

CODE TESTING METHODS

Algorithms (computer). => Is a description of steps that can be undertaken in order to arrive to a solution to a computing problem.

key Attributes of a Computer Algorithm

1. Must be finite (have an endpoint)

2. Must be logical (should make sense)

3. Must be in a specific order (what ought to be done first must appear as first thing, that to be done at the end should appear at the end)

Tools Used in Implementing Algorithms (methods/ technique)

- There are two key tools that a programmer uses to implement a computer Algorithm

Pseudocode

Flowcharts

1. **Pseudocode** => Means false code (pseudo means false.)

- Is a set of English like structured statements used to implement computer algorithms

- in short these are short english like statements for implementing algorithms **// is a tool that is used for implementing a computer argorithm.**

properties of pseudocode

- Statements must be short. (remove all joining words i.e. the, is >>.)

- Statements must not span more than 1 line.

- All action statements must have word that are in block letters.

- all algorithm construct stages words must be given in block letters.

- Statements must be written in the syntax of any programming language.

- Each step/ Statement must be numbered

example. 1

pseudocode for implementing an area of a rectangle

1. DECLARE length, Width, Area.

2. INITIALIZE length.

3. INITIALIZE width.

4. CALCULATTE area.

5. DISPLAY area.

example 2.

1. SET length to ();

2. SET width to ()

3. SET Area to ()

4. READ lenth

5. READ width

6. Area = length x width

7. OUTPUT Area.

KEYWORDS USED FOR EXECUTING CERTAIN PROGRAMMING STAGES AND CONSTRUCTS.

1. words for initialization

- **INIT**
- **INITIALIZE**
- **SET**

2. words used for input stages

- **CAPTURE**
- **GET**
- **READ**

3. words used for output stages

- **OUTPUT**
- **DISPLAY**
- **WRITE**
- **PRINT**

4. words used for selection stages

- **IF** (condition)**THEN**

    //// pseudocode

 ENDIF

5. words used with iteration.

a. for construct;-

- **FOR**

  /////// pseudocode here

  END FOR

b. while construct;

- **WHILE**

  /////// pseudocode here

  END WHILE

c. do ---- while;

- **DO**

  ///////pseudocode here

  END DO

PSEUDOCODE FOR AN ALGORITHM FOR DISPLAYING ALL ELEMENTS IN AN ARRAY.

1. **FOR** i = 0 to l = Array.length – 1 (since array.length goes 1 less the array size)
2. **WRITE** Arrayp[i]
3. **ENDFOR**

PSEUDOCODE FOR AN ALGORITHM WHICH WILL CALCULATE ALL ELEMENTS IN AN ARRAY.

1. SET Sam to 0
2. FOR i = 0 to I = Array.length – 1 (since array.length goes 1 less the array size)
3. ADD Array[i] to Sam
4. ENDFOR
5. PRINT Sam

PSEUDOCODE FOR AN ALGORITHM OF A PROGRAM WHICH WILL DERTEMINE THE LARGEST NUMBER IN AN ARRAY

1. SET Max = Array[0]
2. FOR i = 1 to Array.length – 1
3. IF Array[i] > Max THEN
4. SET Max to Array[i] // Max = Array[i]
5. ENDIF
6. END FOR
7. PRINT Max

Q1. *An array contains the value Y, write a pseudocode search for the value in the Array & Display it.*

Q2. *Write a pseudocode for displaying all values in a 5X5 2-D Array.*

Q3. Write a pseudocode to check for common value in a data set of 2 Array.

Q4. *Write a pseudocode which calculate the average of the grade values in an Array.*

Q5. *Write a pseudocode to determine the smallest numeric value in an array.*

FLOW CHARTS => *is a tool for implementing s computer algorithm*

- Is a diagrammatic representation of the flow of a computer algorithm.

PROPERTIES OF FLOW CHARTS

- Geometric figures are used to specify steps in an algorithm.
- Numbering of steps is not used.
- Each flow chart starts with the START key
- Each flow chart terminates with the STOP keyword
- Words for stages of execution (i.e. Action words) are written inside Symbols => the geometric figure
- Flow chart flows in one direction only with branching for selection points and iteration points

FLOWCHART SYMBOLS

1. The START / STOP symbol.
   - ⇨ Denoted as a round ended rectangle as shown below



=> used for indicating starting point.



=> Used to terminate the point.

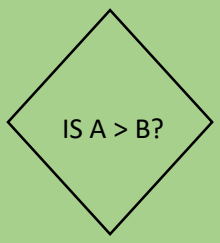
`

2. The PROCESS symbol.
   - ⇨ Is denoted by a normal rectangle as shown below.



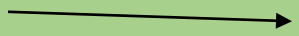
=> it's used to indicate stages that require some processing.

3. the DECISION symbols
   - ⇨ denoted as a diamond as shown below



=> used for indicating stages of an algorithm that require selection programming

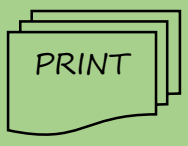
4. the CONNECTOR
   - ⇨ A straight line with a arrow head. To indicate direction of an algorithm flow and connecting chart symbols.



   - The given are key flowchart symbols but there are also others symbols but not frequently used.
     E.g.



=> Hard symbol copy document printing.



=> Hard copy multiple documents printing.

CODE TRACING,

- Code tracing is the art of going though the source code manually in order to test the execution of the code.
- This implies that code tracing is a method for checking the execution of a piece of code before compilation

ADVANTAGES

- Helps to test the execution logic/ order of a program.
- Helps identify bugs early through enough before the code is compiled.
- Gives an insight to code steps that have been poorly designed.

CODE TRACING METHODOLOGIES

- There are several methods used for code tracing, most popular of which is a ***trace table.***

CODE TRACING TOOLS

- Simple tools like
  I. Pencil
  II. Pen
  III. Paper
  IV. Eraser

THE TRACE TABLE

- Is a table that is used as a methodology for implementing code tracing.

HOW TO TRACE AN ALGORITHM USING A TRACE TABLE

I. All steps in a source code /pseudocode (i.e. algorithm) should be numbered
II. Create a table (columns and Rows) which is also numbered.
III. Each step in the pseudocode/ source code must be presented in s unique row in the trace table.
IV. All variables in the source code / pseudo code must be given as column Names.
V. All process stages must be given as column Name in the Trace Table.
VI. All conditions must be given as column Names in a Trace Table.
VII. All looping stages must be given as column Names in a Trace Table.
VIII. The final Stage of the Algorithm is given as output column.

EXAMPLE OF A TRACETABLE

- Algorithm of an area of rectangle program,

Sn.	Length	Width	Area	Area=LXW	output
1	0				
2		0			
3			0		
4	20				
5		10			
6				20 X 10	
7					200

Trace table for a while loop which implements the first five natural numbers

sr	x	X<5?	X++	output
1	0			
2		yes		
3				0
4			1	
5	1			
6		yes		
7				1
8			2	
9	2			
10		yes		
11				2
12			3	
13	3			
14		yes		
15				3
16			4	
17	4			
18		yes		

19				4
20			5	
21	5			
22		no		
23				------

assignment

Sr.	X[i]	i	I < x length	i++	output
1		0			
2			yes		
3					7
4				1	
5	7				
6		1			
7			yes		
8					10
9				2	
10	10				
11		2			
12			yes		
13					-100
14				3	
15	-100				
16		3			
15			yes		
16					20
17				4	

18	20				
19		4			
20			yes		
21					25
22				5	
23	25				
24		5			
25			no		
26					---
27				----	

Ass 2.

POINTERS AND VALUES

- A pointer is a **variable** which holds the address of the location of another variable / data structure in memory.

  Examples of programming languages that us pointer [**c, c++**]

  Declaration of pointers

  Datatype * PointerName;

  e.g. int * p;

  creates a pointer variable called **p**

  mainly pointers are declared as integers.

  Assigning values to variables addresses into pointers

  This is done as follows:

  1. Create a variable
  2. Create the pointer

3. Assign the address of the variable into the pointer using the syntax Pointer = &variableName.

   Example;

   1. Int x;
   2. Int *p;
   3. P = &**x;**

      Step 3 assigns the address of the location of variable x in the RAM [memory].

      P will get the value of the location of x in the memory.

VALUES

- This is the actual value stored in a pointer and is used as an address of the location of a variable in memory

PROGRAMMING PARADIGMS

- Also known as the **programming approach**
- Programming languages are classified according to their approach.
- In computer programming a paradigm, is;
  - ***i.*** Approach
  - ***ii.*** Style
  - ***iii.*** Mode
  - ***iv.*** Technique
  - ⇨ A paradigm is an approach which can be used by a programmer in order to come up with a program.

TYPES OF PROGRAMMING PARADIGMS

- There are many programming paradigms at the moment but the most popular ones are;

i. Imperative programming paradigm.
ii. The functional programming paradigm.
iii. The object-oriented programming paradigm.

- There are others e.g. Logic Programming paradigm [uses the mathematical logic.]
- All programming languages are therefore classified in a programming paradigm which they support.
- Some languages support one programming paradigm
  [e.g. Smalltalk => supports object-oriented programming]
  [Haskell => supports functional programming paradigms]
- Such type of languages is called **single paradigm programming languages**.
- Other languages support **multiple paradigms**
  [e.g. java => supports imperative paradigm, object-oriented paradigms]
  [c, c++]

THE IMPERATIVE PARADIGM

⇨ This is a programming approach in which a program is developed as a set of statements also known as **commands** which are executed in some specific order to change the status / state of the program?

Examples of languages that use the imperative approach

i. C
ii. C++
iii. Java
iv. Php
v. Ruby on rails

FUNCTIONAL PARADIGM

⇨ Focuses on the evaluation of the mathematical functions as the development means of programs

- To come up with a functional program we have to write mathematical instructions or statements

EXAMPLES;

i. Haskell
ii. Python
iii. JavaScript
iv. java

KEY CONCEPT OF THE FUNCTIONAL PROGRAMMING PARADIM

1. Data is immutable (change)
2. State is also immutable
3. Function can only generate a single set of input
   Since the data is mutable there will no be loops or if statement

OBJECT-ORIENTED PROGRAMMING PARADIGM

⇨ Is based on an object

⇨ The objects in **O.O.P** has the following elements

i. **Behaviors** => methods or actions
ii. **Attributes** => properties or identity
iii. **Class** => in which the methods and actions are specified
iv. Exchanging of messages or interaction

- Hence the object is a data structure containing data and methods that work upon the data.
- In OOP paradigm

i. Data is mutable
ii. State is mutable

OBJECT ORIENTED PROGRAMMING

- Is a programming paradigm that focuses on objects and classes in order to develop a software program.
- OOP models real world objects to come up with software by considering the following key features.
  i. Objects in the real world have got property
  ii. ========== behavior [can do action]
  ⇨ So an object in OOP can be able to possess **properties** and **behaviors**
  ⇨ Properties are also known as fields or member fields and behavior are also known as activities/ functions/ sub routine/ member methods.
- Therefore, an object is a bundle of
  i. Data => contained in members or fields
  ii. Methods =>contained in functions
  ⇨ Methods are operations that can be applied on data.

CLASS IN OOP

- Is a template or a blue print that specifies;
  i. Characteristics [attributes]
  ii. Behaviors [actions/ methods]

OBJECTS

- Is a member of a class which inherits the characteristics and behaviors specified to the class.

Difference between a class and an object

i. A class has no physical presence while an object has physical presence.
ii. Out of one class we can generate many objects.
iii. A class is permanent while an object is temporary

iv. A class hardly changes while an object frequently changes based on the environment.

INSTANCE

- This is when a class acquires the characteristics and methods that are found in another class.
- The class containing the characteristics and methods to be acquired by another class in known as the parent class [**base or primary class**]
- The class which acquires the characteristics and behaviors of the parent class is know as the child class [**derived class or class secondary**]

Advantages of instance.

i. Code is reused many times
ii. Helps to develop programs faster.

Disadvantage of an instance

i. When the parent class has been affected, the child class will be affected as well since it will depend on the parent class.

ENCAPSULATION

- Also known as data hiding.
- This is a feature/ approach in which data and operations that works on the data are bundled up together and hidden from other objects/ classes in order to in hence security.
- They are made private from other classes.
- Access to the implementation details of the class/ object is restricted while the functionality details are given.

Advantages.

i. Increases the security of objects from outside attacks or unauthorized access.

POLYMORPHISM

Poly = many

Morphie = terms

- Is a feature which a particular operation can be implemented in many ways
- Is a feature which a method (function) get applied in different ways depending on the object which is using the method.

Types of polymorphism

1. **Compile time polymorphism**

- Also known as **static polymorphism**
- The implementation of the form it will take is done during compile time

NB: compile time is the time where source code is being translated to machine code by a compiler.

2. **Run-time polymorphism**

- Run time is the time when the program is being executed [is being used by an end-user]
- Run-time polymorphism is also known as **dynamic polymorphism**
- This is when the form of the method it will take is decided at run-time or as the program is executing.

DYNAMIC DISPATCH

- Is a feature which a method implementing [as regards to the fore it will take] is decided at run-time [e.g. gets differed to run-time]

OBJECT ORIENTED DESIGN CONSIDERATIONS.

- These are principles that which should be followed in the designing of an OO program in order to ensure
  i. Quality of code
  ii. Security of the software developed
  iii. Performance of the software
  iv. Extensibility of the software
  v. Maintainability of the software.

OBJECT ORIENTED PROGRAMMING CONSIDERATIONS

COUPLING

- Refers to the interdependency between classes in a software.

Types of coupling

1. **Strong coupling**.
- This is when a class **heavily** depends on another class in order to enable it to give service.

2. **Wear coupling.**
- This is when a class is not strongly depended on another class [**it is loosely coupled to another class**].
  - Which class is desirable in OOP?
  - Wear coupling [].

COHESION

- Cohesion is the level of focus of a class which is implementing and accomplishing a single task (how much focused the methods is, a class are as regards to accomplishing a specific task.)

Types of cohesion

i. **High cohesion**

- This is an OOP design consideration to which a class is developed with methods that focus in accomplishing the design of the class.

ii. **Low cohesion**

- This is one in which a class is poorly design such that methods that method that are not inline with the design purpose find their way into the class.
  - Which design is desirable in OOP?
    - High cohesion.

VISIBILITY MODIFIERS

- There are also known as access modifiers.
- They are keywords that are used to describe the accessibility of;

i. Variables
ii. Classes
iii. Methods

Types of visibility modifiers

1. The public modifier.

- this is used for modifying the accessibility of a class, method and variables making them accessible and visible to all entities in a program (objects/ classes)

e.g., public class Login

NB public modifier is developed by a [+] sign in the class diagrams.

2. The private modifier

- This is a modifier which restricts the access of members to class from outside

NB: no object/ class can access a private method.

- Only members of that particular class have the access to a private method
- The private keyword is used to describe a private class method

e.g. private class Login

the [-] symbol is used to denote a private method in the class diagrams

+ Login
+ username: string + password: int
_ submit ()
_ validate ()